

Rua 使用手册

v1.1.0

bzhao

2025-04-03

目录

1 mkinfo	3
1.1 用法	3
1.2 示例	3
2 compdb	4
2.1 生成编译数据库(Gen)	4
2.1.1 使用默认方法(built-in)生成编译数据库	4
2.1.2 使用 intercept-build 方法生成编译数据库	4
2.1.3 FQA	4
2.2 归档编译数据库(Add)	4
2.3 删除编译数据库(Del)	4
2.4 列出编译数据库(Ls)	4
2.5 选择编译数据库(Use)	4
2.6 命名编译数据库(Name)	4
2.7 备注编译数据库(Remark)	4
3 showcc	5
3.1 用法	5
3.2 示例	5
4 review	6
4.1 用法	6
5 perfan	7
5.1 用法	7
5.2 示例	7
5.3 输出格式	7
5.4 工具比较	7
6 clean	8
6.1 用法	8
6.2 示例	8
7 init	8
7.1 用法	8
7.2 示例	8

1 mkinfo

rua mkinfo 用于生成目标平台的构建指令。该指令包含了众多常用的 make 变量，用于方便地定制构建行为。例如：通过传入 -6/--ipv6 选项，令构建目标中包含 ipv6 字样；通过传入 -w/--webui 选项，将在指令中的 NOTBUILDWEBUI 变量设置为 0（带 WebUI）。

1.1 用法

用户可通过 rua mkinfo -h 查看帮助信息：

```
> rua mkinfo -h
Get all matched makeinfos for product

Usage: rua mkinfo [OPTIONS] <NAME-OR-TARGET>

Arguments:
  <NAME-OR-TARGET>  Product name such as A1000, or build target (with --target switch on) such as a-dnv, View as a product name by default. Regex is also supported when using as a product name, e.g. 'X\d+80'

Options:
  -4, --ipv4          Build with only IPv4 enabled
  -6, --ipv6          Build with IPv6 enabled
  -g, --coverage      Run coverage
  -c, --coverity       Run coverity
  -d, --debug          Build in debug mode (default is release mode)
  --format <FORMAT>  Output format for makeinfos, defaults to list [default: list] [possible values: csv, json, list, tsv]
  -p, --password      Build with shell password enabled
  -w, --webui          Build with WebUI enabled
  -s, --image-server <IMAGE-SERVER> Server to upload the output image to [possible values: b, s]
  --nostrip <BINARY> Binaries that get out of strip processing
  --by-target          Treat the positional arg as a build target other than a product name
  -h, --help          Print help

Examples:
  rua mkinfo A1000      # Makeinfo for A1000 without extra features
  rua mkinfo -6 A1000   # Makeinfo for A1000 with IPv6 enabled
  rua mkinfo -6w 'X\d+' # Makeinfos for X-series products with IPv6 and WebUI enabled using regex pattern
  rua mkinfo --target a-dnv # Makeinfos for a-dnv target
```

1.2 示例

1. rua mkinfo -6 A1000：生成 A1000 平台的构建指令，启用 IPv6 支持

```
bzhao in build18-bj in ~/repos/REL_R11
> rua mkinfo -6 A1000
1 matched info:
-----
Product   : SG-6000-A1000
Model     : HS_TYPE_SG6000_A1000
Family    : HS_PRODUCT_FAMILY_FIREWALL
Platform  : HS_A1000
Target     : a-dnv-ipv6
Directory : products/ngfw_as/
Command   : hsdocker7 "make -C products/ngfw_as/ -j8 a-dnv-ipv6 ISBUILDRELEASE=1 NOTBUILDUNIWEBUI=1 HS_SHELL_PASSWORD=0 HS_BUILD_COVERAGE=0 HS_BUILD_COVERITY=0 OS_IMAGE_FTP_IP=10.200.6.10 IMG_NAME=SG6000-REL_R11-ADNV-V6-r0215-bzhao >build.log 2>&1"
-----
Run make command under the project root, i.e. "/home/bzhao/repos/REL_R11"
```

2. rua mkinfo -w A1000：生成 A1000 平台的构建指令，启用 WebUI 支持

```
bzhao in build18-bj in ~/repos/REL_R11
> rua mkinfo -w A1000
1 matched info:
-----
Product   : SG-6000-A1000
Model     : HS_TYPE_SG6000_A1000
Family    : HS_PRODUCT_FAMILY_FIREWALL
Platform  : HS_A1000
Target     : a-dnv
Directory : products/ngfw_as/
Command   : hsdocker7 "make -C products/ngfw_as/ -j8 a-dnv ISBUILDRELEASE=1 NOTBUILDUNIWEBUI=0 HS_SHELL_PASSWORD=0 HS_BUILD_COVERAGE=0 HS_BUILD_COVERITY=0 OS_IMAGE_FTP_IP=10.200.6.10 IMG_NAME=SG6000-REL_R11-ADNV-V6-r0215-bzhao >build.log 2>&1"
-----
Run make command under the project root, i.e. "/home/bzhao/repos/REL_R11"
```

3. rua mkinfo -6w A1000：生成 A1000 平台的构建指令，启用 IPv6 支持以及 WebUI 支持

```
bzhao in build18-bj in ~/repos/REL_R11
> rua mkinfo -6w A1000
1 matched info:
-----
Product   : SG-6000-A1000
Model     : HS_TYPE_SG6000_A1000
Family    : HS_PRODUCT_FAMILY_FIREWALL
Platform  : HS_A1000
Target     : a-dnv-ipv6
Directory : products/ngfw_as/
Command   : hsdocker7 "make -C products/ngfw_as/ -j8 a-dnv-ipv6 ISBUILDRELEASE=1 NOTBUILDUNIWEBUI=0 HS_SHELL_PASSWORD=0 HS_BUILD_COVERAGE=0 HS_BUILD_COVERITY=0 OS_IMAGE_FTP_IP=10.200.6.10 IMG_NAME=SG6000-REL_R11-ADNV-r0215-bzhao >build.log 2>&1"
-----
Run make command under the project root, i.e. "/home/bzhao/repos/REL_R11"
```

4. rua mkinfo -ss A1000：生成 A1000 平台的构建指令，上传到 10.200.6.10 服务器，即苏州服务器

```
bzhao in build18-bj in ~/repos/REL_R11
> rua mkinfo -s s A1000
1 matched info:
-----
Product   : SG-6000-A1000
Model     : HS_TYPE_SG6000_A1000
Family    : HS_PRODUCT_FAMILY_FIREWALL
Platform  : HS_A1000
Target     : a-dnv
Directory : products/ngfw_as/
Command   : hsdocker7 "make -C products/ngfw_as/ -j8 a-dnv ISBUILDRELEASE=1 NOTBUILDUNIWEBUI=1 HS_SHELL_PASSWORD=0 HS_BUILD_COVERAGE=0 HS_BUILD_COVERITY=0 OS_IMAGE_FTP_IP=10.200.6.10 IMG_NAME=SG6000-REL_R11-ADNV-r0215-bzhao >build.log 2>&1"
-----
Run make command under the project root, i.e. "/home/bzhao/repos/REL_R11"
```

5. rua mkinfo --format json A1000：生成 A1000 平台的构建指令，输出格式指定为 JSON 格式，适合脚本使用

```
bzhao in build18-bj in ~/repos/MX_MAIN
> rua mkinfo --format json A1000
[
  {
    "Command": "hsdocker7 \"make -C products/ngfw_as/ -j8 a-dnv ISBUILDRELEASE=1 NOTBUILDUNIWEBUI=1 HS_SHELL_PASSWORD=0 HS_BUILD_COVERAGE=0 HS_BUILD_COVERITY=0 OS_IMAGE_FTP_IP=10.200.6.10 IMG_NAME=SG6000-MXMAIN-ADNV-r0331-bzhao >build.log 2>&1\"",
    "Directory": "products/ngfw_as/",
    "Family": "HS_PRODUCT_FAMILY_FIREWALL",
    "Model": "HS_TYPE_SG6000_A1000",
    "OEMID": "HS_OEMID_BJHS",
    "Platform": "HS_A1000",
    "Product": "SG-6000-A1000",
    "Target": "a-dnv"
  }
]
```


2 compdb

rua compdb 包含了众多子命令，分别用于生成、删除和管理编译数据库。

编译数据库能够为 C/C++ 语言服务器 (Language Server, LS) 提供各编译单元的编译指令，如 include 路径、编译宏，使之能够理解每个文件的编译方式，从而在代码代码库中正确地行走。

编译数据库的格式为 JSON 格式，文件名为 compile_commands.json，存放在当前工作目录下。该文件是一个数组，每个元素对应一个编译单元的编译指令。

编译数据库的每个元素包含了以下几个字段：

- directory：编译单元所在的目录
- command：编译单元的编译指令
- file：编译单元的文件路径

一般来说，编译数据库是分构建目标的，如 a-dnv-ipv6 对应有一个编译数据库，a-dnv 对应有另一个编译数据库。对于后者而言，IPv6 宏所包裹的代码不会被解析，LS 视之为无效代码。

Rua 工具为每条命令和参数添加了充分的注释，当存在疑惑时请使用 -h 或 --help 参数查看帮助信息。例如，工具顶层命令帮助信息可通过在 shell 下执行 rua compdb -h 查看：

```
> rua compdb -h
Manipulate compilation database

Usage: rua compdb <COMMAND>

Commands:
  gen      Generate a JSON compilation database (JCDB) for the given target
[aliases: generate]
  add      Archive the currently used compilation database into store as a new generation [aliases: ark, archive]
  del      Delete compilation database generation(s) from store [aliases: delete, rm, remove]
  ls       List all compilation database generations in store [aliases: list]
  use      Select a compilation database generation from store to use
  name     Name a compilation database generation
  remark   Remark a compilation database generation
  help     Print this message or the help of the given subcommand(s)

Options:
  -h, --help  Print help
```

2.1 生成编译数据库 (Gen)

```
> rua compdb gen -h
Generate a JSON compilation database (JCDB) for the given target

Usage: rua compdb gen [OPTIONS] <PATH> <TARGET>

Arguments:
  <PATH>      Path for the target where platform-specific makefiles reside, such as 'products/vfw'
  <TARGET>    Target to build, such as 'a-dnv'

Options:
  -D, --define <KEY=VAL>
                    Define a variable which will be passed to the underlying make command
  -e, --engine <ENGINE>
                    Engine for generating compilation database (defaults to built-in)
[possible values: built-in, intercept-build, bear]
  -b, --bear-path <BEAR>
                    Path to the bear binary (defaults to /devl/sw/bear/bin/bear)
  -i, --intercept-build <INTERCEPT-BUILD>
                    Path to the intercept-build binary (defaults to /devl/sw/llvm/bin/intercept-build)
  -h, --help
                    Print help (see more with '--help')
```

Examples:

```
rua compdb gen products/ngfw_as a-dnv # For A1000/A2000...
rua compdb gen products/ngfw_as a-dnv-ipv6 # For A1000/A2000...
with IPv6 support
rua compdb gen -e intercept-build products/ngfw_as a-dnv # For A1000/A2000...
using intercept-build
rua compdb gen . a-dnv # For A1000/A2000...
under submod dir
rua compdb gen -e bear . a-dnv # For A1000/A2000...
under submod dir using bear
run compdb gen -e intercept-build . a-dnv # For A1000/A2000...
under submod dir using intercept-build
```

Caution:

Some files are modified while running in built-in mode which is the default and faster:

1. When running under project root dir:
 - scripts/last-rules.mk
 - scripts/rules.mk
 - Makefile
2. When running under submod dir:
 - scripts/last-rules.mk
 - scripts/rules.mk

These files may be left dirty if compdb process aborted unexpectedly. You could manually restore them by execute:

```
svn revert Makefile scripts/last-rules.mk scripts/rules.mk
```

参数解析：

- <PATH>：构建路径，例如 A1000/A2000 平台的构建路径为 products/ngfw_as，K6580 平台的构建路径为 products/ngfw_ak OR ...
- <TARGET>：构建目标，例如 A1000/A2000 等平台的编译目标为 a-dnv，K6580 平台的编译目标为 hygon，X8180 平台的编译目标为 tai
- -e/-engine：可选，引擎。候选值为 built-in、intercept-build、bear，默认值为 built-in
- -D/-define：可选，变量定义。将会传递给底层的 make 命令
- -b/-bear-path：可选，指定 bear 的路径，默认值为 /devl/sw/bear/bin/bear
- -i/-intercept-build-path：可选，指定 intercept-build 的路径，默认值为 /devl/sw/llvm/bin/intercept-build

2.1.1 使用默认方法(built-in)生成编译数据库

1. 在工程根目录下生成一个编译目标为 kunlun-ipv6 的编译数据库：

```
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb gen products/ngfw_as kunlun-ipv6
[2/2] Injecting makefiles (/home/bzhao/repos/MX_MAIN/scripts/last-rules.mk & /home/bzhao/repos/MX_MAIN/scripts/rules.mk & /home/bzhao/repos/MX_MAIN/Makefile modified)...
[2/2] Building makefiles [00:00:00]
docker image name: hu_1406_make_20250208
start docker containerid:6360e7b041ef
[2/2] Building makefiles [00:00:00]
groupadd: group 'huw' already exists
useradd: user 'bzhao' already exists
[2/2] Building makefiles [00:04:01]
[2/2] Building makefiles [00:04:01]
[2/2] Building makefiles [00:04:01]
[2/2] Restoring makefiles (/home/bzhao/repos/MX_MAIN/scripts/last-rules.mk & /home/bzhao/repos/MX_MAIN/scripts/rules.mk & /home/bzhao/repos/MX_MAIN/Makefile restored)...
[4/2] Parsing building...
[4/2] Generating JCDB...
Archiving the newly generated compilation database to store...ok
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb ls
Generation Branch Revision Target Date Name Remark
26 MX MAIN 309522 tai 2025-02-31T11:04:54
27 MX MAIN 307164 a-dnv-ipv6 2025-02-24T15:55:14 A Compilation database for A
```

2. 在工程根目录下为 X8180 平台生成一个编译数据库，编译目标为 tai：

```
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb gen products/ngfw_ak tai
[2/2] Injecting makefiles (/home/bzhao/repos/MX_MAIN/scripts/last-rules.mk & /home/bzhao/repos/MX_MAIN/scripts/rules.mk & /home/bzhao/repos/MX_MAIN/Makefile modified)...
[2/2] Building makefiles [00:00:00]
docker image name: hu_1406_make_20250208
start docker containerid:6360e7b041ef
[2/2] Building makefiles [00:00:00]
groupadd: group 'huw' already exists
useradd: user 'bzhao' already exists
[2/2] Building makefiles [00:04:01]
[2/2] Building makefiles [00:04:01]
[2/2] Building makefiles [00:04:01]
[2/2] Restoring makefiles (/home/bzhao/repos/MX_MAIN/scripts/last-rules.mk & /home/bzhao/repos/MX_MAIN/scripts/rules.mk & /home/bzhao/repos/MX_MAIN/Makefile restored)...
[4/2] Parsing building...
[4/2] Generating JCDB...
Archiving the newly generated compilation database to store...ok
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb ls
Generation Branch Revision Target Date Name Remark
26 MX MAIN 309522 tai 2025-02-31T11:04:54
27 MX MAIN 309522 kunlun-ipv6 2025-02-31T11:04:54
28 MX MAIN 307164 a-dnv-ipv6 2025-02-24T15:55:14 A Compilation database for A
```

2.1.2 使用 intercept-build 方法生成编译数据库

intercept-build 是 LLVM 工具集所提供的工具，能够拦截编译命令并生成编译数据库。

由于该工具的工作原理为拦截每次触发编译器编译的指令，因此不适合增量编译场景。增量编译场景下，编译器只会编译那些有变更的文件，而不会编译所有文件。这样一来，intercept-build 就无法捕获到所有的编译指令。因此，建议在全量编译时使用该工具。

2.1.3 FQA

1. rua compdb gen 执行失败怎么办？
命令执行失败的原因有以下几个：
 - 无执行权限，建议使用 chmod +x <RUA-PATH> 添加可执行权限
 - 执行目录不正确，建议在工程根目录下执行
 - Makefile 状态不正确，简易检查 “scripts/rules.mk”、“scripts/last-rules.mk”、“Makefile” 三个文件是否被修改过。若有修改，建议执行 svn revert 恢复
 - 磁盘满了，建议检查磁盘空间是否足够。该类错误不但会导致编译数据库生成失败，还可能造成 “scripts/rules.mk”、“scripts/last-rules.mk”、“Makefile” 三个文件被修改
2. 无法生成第三方库的编译数据库怎么办？
rua compdb gen 默认使用 built-in 方法，该方法只生成我们的内部代码，暨通过 “scripts/last-rules.mk” 中一条用于编译目的的 recipe 来编译的文件。
若想单独生成第三方库的编译数据库，可使用 bear 或 intercept-build 方法。
若想使用 bear 或 intercept-build 方法生成整个工程的编译数据库，须在一个 clean 过的分分支下进行（从而进行全量编译），或者编译一个当前未被 ccache 缓存其编译信息的编译目标，否则会失败。

2.2 归档编译数据库(Add)

归档功能适合将其他工具生成的编译数据库归档到 store 中，以便管理。其用法如下：

```
> rua compdb add --help
Archive the currently used compilation database into store as a new generation

Usage: rua compdb add [OPTIONS] <TARGET>

Arguments:
  <TARGET>    Target specified for the compilation database

Options:
  -r, --revision <REVISION>
                    Revision for compilation database (defaults to current repo revision)
  -f, --compilation-database <COMPILATION-DATABASE>
                    Use this compilation database other than the default (compile_commands.json)
  -h, --help
                    Print help

Examples:
  rua compdb add hygon-ipv6 # Archive compilation database for hygon-ipv6
  rua compdb add --revision 307164 hygon # Archive compilation database for hygon with a revision provided
```

参数解析：

- <TARGET>：必填，用于指示编译数据库所对应的构建目标，如 a-dnv、hygon
- -r/-revision：可选，用于指示编译数据库所对应的代码版本，默认使用当前工作目录的代码版本
- -f/-compilation-database：可选，用于手动指定编译数据库路径，默认使用当前工作目录下的 compile_commands.json

例如：

将当前工作目录下的 compile_commands.json 添加到 store 中：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-14T17:59:44
2 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-14T17:59:13
1 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A Compilation database for A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb add a-dnv
Archiving compilation database for into store...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 a-dnv 2025-02-14T18:05:51
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-14T17:59:44
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-14T17:59:13
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A Compilation database for A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

2.3 删除编译数据库 (Del)

```
> rua compdb del -h
Delete compilation database generation(s) from store

Usage: rua compdb del [OPTIONS] [GENERATION-ID]

Arguments:
  [GENERATION-ID]  Generation to delete

Options:
  -a, --all          Remove all generations
  -n, --new <N>     Remove N newest generations
  -o, --old <N>     Remove N oldest generations
  -h, --help         Print help
```

例如：

1. 删除 store 中的 Generation 2：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:59:28
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-16T20:52:54
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-16T20:59:07
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb del 2
Deleting generation 2...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:59:28
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-16T21:03:23
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:52:54
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

2. 删除 store 中最旧的 3 个编译数据库：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:53:23
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-16T20:52:54
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-16T20:59:07
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:52:36
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb del -o 3
Deleting 3 oldest generations...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T20:53:23
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-16T20:52:54
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-16T21:03:26
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T21:03:06
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

3. 删除 store 中较新的两个编译数据库：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-14T18:05:51
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-14T17:59:44
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-14T17:59:13
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A Compilation database for A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb del --new 2 # 删除最新的两个 generations
Deleting 2 newest generations...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-14T17:59:13
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-11T10:04:58 A1000-A Compilation database for A1000-A
1 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

4. 删除 store 中所有的编译数据库：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi-ipv6 2025-02-16T21:03:15
2 HAWAII_REL_R10_FR41504_FR39970 306395 a-dnv 2025-02-16T21:03:06
1 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-16T21:03:06
0 HAWAII_REL_R10_FR41504_FR39970 306395 a-dnv-ipv6 2025-02-16T21:03:01
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb del -a
Deleting all generations...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
No compilation database generation available
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

2.4 列出编译数据库 (Ls)

列出当前工作目录下所有的编译数据库。

```
> rua compdb ls -h
List all compilation database generations in store

Usage: rua compdb ls

Options:
  -h, --help  Print help
```

每个表项都有一个唯一的 Generation ID，且关联 4 个重要属性和 2 个可选属性：

- Revision：代码版本
- Target：构建目标
- Date：生成日期
- Name：可选，编译数据库的名字
- Remark：可选，编译数据库的备注

当一个 generation 后面标有黄色的 * 时，表示它是它是当前使用项。gen/use 操作会使当前使用项发生改变，因此 * 的显示会发生改变。需要注意的是，为保证准确性，尽量使用 rua 来生成、切换数据库。

例如：

```
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb ls
Generation Branch Revision Target Date Name Remark
30 MX MAIN 309522 hygon-ipv6 2025-04-03T16:06:06
29 MX MAIN 309522 a-dnv-ipv6 2025-04-01T20:03:36
28 MX MAIN 309522 tai 2025-03-31T14:02:48
27 MX MAIN 309522 kunlun-ipv6 2025-03-31T11:04:54
26 MX MAIN 307164 a-dnv-ipv6 2025-02-24T15:55:14 A Compilation database for A
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb use 30
Switching to generation 30...ok
bzhao in ● build18-bj in ~/repos/MX_MAIN
> rua compdb ls
Generation Branch Revision Target Date Name Remark
30 * MX MAIN 309522 hygon-ipv6 2025-04-03T16:06:06
29 MX MAIN 309522 a-dnv-ipv6 2025-04-01T20:03:36
28 MX MAIN 309522 tai 2025-03-31T14:02:48
27 MX MAIN 309522 kunlun-ipv6 2025-03-31T11:04:54
26 MX MAIN 307164 a-dnv-ipv6 2025-02-24T15:55:14 A Compilation database for A
bzhao in ● build18-bj in ~/repos/MX_MAIN
```

2.5 选择编译数据库 (Use)

选中的编译数据库将会覆盖当前工作目录下的 compile_commands.json 文件。

```
> rua compdb use -h
Select a compilation database generation from store to use

Usage: rua compdb use <GENERATION>

Arguments:
  <GENERATION>  Compilation database generation id

Options:
  -h, --help  Print help
```

2.6 命名编译数据库 (Name)

```
> rua compdb name -h
Name a compilation database generation

Usage: rua compdb name <GENERATION> <NAME>

Arguments:
  <GENERATION>  The compilation database generation
  <NAME>        Name for the compilation database

Options:
  -h, --help  Print help
```

例如：

为 store 中的编译数据库 generation 1 添加一个名字：

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-11T10:04:58
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-11T10:04:58 A1000-A
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb name 1 A1000-A...ok
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-11T10:04:58
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-11T10:04:58 A1000-A
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```

2.7 备注编译数据库 (Remark)

```
> rua compdb remark -h
rua compdb remark <GENERATION-ID> <备注>
Remark a compilation database generation

Usage: rua compdb remark <GENERATION> <REMARK>

Arguments:
  <GENERATION>  The compilation database generation
  <REMARK>      Remark for the compilation database generation

Options:
  -h, --help  Print help
```

例如：

为 store 中的编译数据库 Generation 1 添加备注

```
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-11T10:04:58
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-11T10:04:58 A1000-A
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb remark 1 编译数据库备注
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
> rua compdb ls
Generation Branch Revision Target Date Name Remark
3 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
2 HAWAII_REL_R10_FR41504_FR39970 306395 tai-ipv6 2025-02-11T10:04:58
1 HAWAII_REL_R10_FR41504_FR39970 306395 hygon 2025-02-11T10:04:58 A1000-A 编译数据库备注
0 HAWAII_REL_R10_FR41504_FR39970 306395 tianchi 2025-02-11T10:04:58 A1000-A
bzhao in ● build18-bj in ~/repos/HAWAII_REL_R10_FR41504_FR39970
```


3 showcc

显示某个文件的编译指令，依赖于编译数据库。

3.1 用法

用户可通过 `lua showcc -h` 查看帮助信息:

```
> rua showcc -h
Show all possible compile commands for filename (based on compilation database)
```

Usage: `lua showcc [OPTIONS] <SOURCE-FILE>`

Arguments:

<SOURCE-FILE> Source file name for which to fetch all the available compile commands

Options:

```
-c, --compdb <COMPDB>  Compilation database (defaults to file
"compile_commands.json" in the current directory)
-h, --help                Print help
```

3.2 示例

1. 显示 flow first.c 文件的编译指令:

```
run showenv flow_first.c  
1 matched record:  
  
File      : /home/bzhao/repos/REL_R11/src/d-plane/flow/flow_first.c  
Directory : /home/bzhao/repos/REL_R11/src/d-plane/flow  
Command   : gcc -DOEM_HILLSTONE -Wno-error-unused-but-set-variable -Wno-error-enum-compare -Wno-error-uninitialized -Wno-error-int-to-pointer-cast -Wno-error-unused-but-set-parameter -Wno-error-switch -Wunused-but-set-variable -Wno-error-attributes -Wno-error-deprecated-declarations -Wno-error-address -Wno-error-unused-variable -Wno-error-maybe-uninitialized -march=corei7-avx -mno-avx -mno-avx2 -Wno-error-delete-non-virtual-dtor -Wno-error-narrowing -mno-avx2 -DHS_HYPERSCANAN-I-OZ -fstrict-align -lasng-gcc-W-Wall -Werror -Wno-poiner-sign -Wno-unused-parameter -DHSSERIES-SA-I -fno-optimize-framepointer -DHS_SOFT_VERSION="" -DHS_COPYRIGHT_YEAR=""  
to-init cast -Wno-write-strings -Wno-address -DHS_DPOK_VERSION="" -DDP_HTTPS_I-DIS_GCC_VERSION82 -DHS_LINUX_VERSION504 -DBENMSYS=D STONEOS6I -Wno-pointor-to-int-cast -Wno-write-strings -Wno-address -DCVMX_ENABLE_POW_CHECKMCHB0 -DCVMX_ENABLE_PARAMETER_CHECKING0 -DCVMX_ENABLE_CSR_ADDRESS_CHECKING -O02 -fno-strict-aliasing -fpic -g -w -Wall -Werror -Wno-poiner-sign -Wno-unused-parameter -DHSSERIES-SA-I -DHS_ARCH_X86_64-I -DHS_CPU_X86_64-HYGON-I DXSG_64 HYGO N-I -DHS_LICENSE CHECK-I DHS_DECODE VERSION-I DHS_CHASSIS CHECK-I DBETA CRASH FIXED -DHS_FLOW_MCHECK0 -DHAVE_IPV6-I DHAVE_IPV6 READY-I DHS_STATS-I DMONITOR MEM_S TAT-I DHS_DBGLES DEBUG-I DSNMT BLOCK64-I DHAVE IPV6-I DHAVE IPV6_READY-I DHS_PRODUCT NGFW -DHS_SUPPORT NUMA-DDOPDK FLOW-I DHS_SUPPORT ANKE-I DPLATFORM DPDK MA X QUEUE_NUM144 -DHS_DPOK_JUMB FRAME-I DHS_SUPPORT VRF-I DHS_SUPPORT FPGA FLOW-I -.include ./.include/platform-. /.include/platform/cavium-. /.include/avr-. /.include/avr/jmai-engine-. /.include ./home/bzhao/repos/REL_R11/src/include/mgd./home/bzhao/repos/REL_R11/src/share-include ./home/bzhao/repos/REL_R11/src/sha re-include ./home/bzhao/repos/REL_R11/src/share-include/platform-/home/bzhao/repos/REL_R11/src/libcap-/home/bzhao/repos/REL_R11/src/libscv-/home/bzhao/repos/R EL_R11/src/libexpat-. /ctrl.smp-/home/bzhao/repos/REL_R11/src/libtftpfilter-. /app/infra/sslproxy-/home/bzhao/repos/REL_R11/src/libai-include-. /fpga-/home /bzhao/repos/REL_R11/src/libsample/include-. /sample collect-. /onepoint-. /include/features-. /include/features/apr-/home/bzhao/repos/REL_R11/src/share-include e/dp2ap_mrg-. /fs.flow-. /include/platform/i868-/home/bzhao/repos/REL_R11/src/share-include-/home/bzhao/repos/REL_R11/gshare/include-/home/bzhao/repos/REL_R L11/home/bzha o/repos/REL_R11/fs/arch-include/X86_64_gn-ODIO MI-/home/bzha o/repos/REL_R11/src/share-include/nfyw.ac include-/home/bzha o/repos/REL_R11/src/helinux/include/unipi DATA PLACE-0 GNU SOURCE-. /hs/include-/home/bzhao/repos/REL_R11/src/d-plane/include/mpcu-common-/home/bzhao/repos/RE L11/src/include -Wno-unused-but-set-variable-fPIC -rdynamic-I/home/bzha o/repos/REL_R11/src/include/arch/x86_64-MMD -c-o /home/bzhao/repos/REL_R11/target/stoneso s-X86_64-hylon-ipv6-2.0/src/d-plane/flow_flow_first.o flow_first.c
```

2. 显示 virtual wire.c 的编译指令:

```

1 runa showcc virtual.wire.c
1 matched record:
File      : /home/bzhao/repos/REL_R11/src/d-plane/Flow/virtual.wire.c
Directory : /home/bzhao/repos/REL_R11/src/d-plane/Flow
Options   : -g -DGEN_HUI_HOME -Wno-error-unused-but-set-variable -Wno-error-enump-compare -Wno-error-uninitialized -Wno-error-int-to-pointer-cast -Wno-error-unused-but-set-parameter -Wno-error-switch -Wunused-but-set-variable -Wno-error-attributes -Wno-error-deprecated-declarations -Wno-error-address -Wno-error-unused-variable -Wno-error-maybe-uninitialized -march=corei7-avx -Wno-avx -Wno-avx2 -Wno-error-delete-non-virtual-dtor -Wno-error-narrowing -mno-avx2 -DWS_HYPERCANAL -I -O2 -fno-strict-aliasing -g -W -Wall -Werror -Wno-pointer-sign -Wno-unused-parameter -DHSSERIES_SA1 -fno-omit-frame-pointer -DHS_SOFT_VERSION="" -DSCN_COPYRIGHT_YEAR="" "" "" "" ""
DHS_STONEOS1 -Wno-pointer-to-int-cast -Wno-pointer-to-string -Wno-address -DHS_DPOK_VERSION=1911 -DDPO HTB1-1 DHS_GOC_VERSION=482 DHS_LINUX_VERSION=504 -D_GENKSYS_CFG
DHS_STONEOS1 -Wno-pointer-to-int-cast -Wno-write-strings -Wno-address -DOVMX_ENABLE_POW_CHECKS=0 -DOVMX_ENABLE_PARALLEL_CHECKING=0 -DOVMX_ENABLE_CSP_ADDRESS_CHECKING=0
DHS -O -fno-stack-protector -DGEN_HUI_HOME -Wno-error-unused-but-set-parameter -Wno-error-pointer-sign -Wno-unused-parameter -DHSSERIES_SA1 -DHS_ARCH_X86_64-1 -DHS_CPU_X86_64_HYGON-1 -DX86_64_HYGON
M1 -DHS_LICENSE_CHECK=1 -DHS_DEBUG_VERSION=1 -DHS_CHASSIS_CHECK=1 -DBETA_CRASH_FIXED -DHS_FLOW_MCHECK=0 -DHAVE_IPV6-1 -DHAVE_IPV6_READY-1 -DHS_STATS-1 -DMONITOR_MEM_S
TAT-1 -DHS_GBLRES_DEBUG-1 -DSNAT_BLOCK_64-1 -DHAVE_IPV6-1 -DHAVE_IPV6_READY-1 -DHS_PRODUCT_NGFW -DHS_SUPPORT_NUMA -DDPOK_FLOW-1 -DHS_SUPPORT_ANKE-1 -DPLATFORM_DPOK_MA
X_XQUE_NUM144 -DHS_DPOK_JUMBO_FRAME-1 -DHS_SUPPORT_VRF-1 -DHS_SUPPORT_FPGA_FLOW-1 -I./include -I./include/platform -I./include/platform/cavium -I./include/av -I
./include/av/jma4-engine -I././include -I/home/bzhao/repos/REL_R11/src/include/mgd -I/home/bzhao/repos/REL_R11/src/share-include -I/home/bzhao/repos/REL_R11/src/sha
re-include -I/home/bzhao/repos/REL_R11/src/share-include/platform -I/home/bzhao/repos/REL_R11/src/libcap -I/home/bzhao/repos/REL_R11/src/libscv -I/home/bzhao/repos/R
EL_R11/src/libscv/include -I/home/bzhao/repos/REL_R11/src/libsecp256k1 -I./api/infer/edsl/edslrow -I/home/bzhao/repos/REL_R11/src/libalib/include -I./api/
bzhao/repos/REL_R11/src/libasample/include -I./sample_collect -I./honeyopt -I./include/feature -I./include/feature/ap -I/home/bzhao/repos/REL_R11/src/share-include
d/d2ap2asm -I./fsf_flow -I./include/platform/i686 -I/home/bzhao/repos/REL_R11/src/share-include -I/home/bzhao/repos/REL_R11/gshare/minclude -I/home/bzhao/repos/REL_R
11/src/share-include/x86_64 -I/home/bzhao/repos/REL_R11/src/share-include/x86_64_mi -DGPU_M1 -I/home/bzhao/repos/REL_R11/src/share-include/ngfw_ak_include -I/home/bzha
o/repos/REL_R11/src/hslinux/include/uapi -DDATA_PLANE -DGNU_SOURCE -I./ha/include -I/home/bzhao/repos/REL_R11/src/d-plane/include/mcpu-common -I/home/bzhao/repos/REL
R11/src/include -Wno-unused-but-set-variable -EPIC -DYNAMIC -I/home/bzhao/repos/REL_R11/src/include/arch/x86_64 -DMD -c -o /home/bzhao/repos/REL_R11/target/stoneos
x86_64-hYGON-IPv6-2.0/src/d-plane/flow/virtual.wire.o virtual.wire.c
Run compile command under corresponding directory.

```

4 review

该子命令与 autoreview-cops 工具相似，参数更符合直觉。

4.1 用法

```
• [podman] > rua review -h
Start a new review request or refresh the existing one if review-id provided

Usage: rua review [OPTIONS] --bug <BUG> [FILE]...

Arguments:
  [FILE]...  Files to be reviewed

Options:
  -n, --bug <BUG>          Bug id for this review request (required)
  -r, --review-id <REVIEW-ID> Existing review id
  -d, --diff-file <DIFF-FILE> Diff file to be used
  -u, --reviewers <REVIEWERS> Reviewers
  -b, --branch <BRANCH>    Branch name for this commit
  -p, --repo <REPO>        Repository name
  -s, --revision <REVISION> Revision to be used
  -t, --template-file <TEMPLATE-FILE> Use customized template file (please
ensure it can run through svn commit hooks)
  -h, --help                Print help
```

5 perfan

rua perfan 用于对 profiling text 进行指令地址映射。

5.1 用法

Extensively map instructions to file locations (inline expanded)

Usage: rua perfan [OPTIONS] <FILE>

Arguments:
<FILE> Profiling text generated by perfan

Options:
-o, --format <FORMAT> Output format [default: table] [possible values: json, table]
-e, --elf <ELF> Binary files used for addresses resolving [aliases: exe, executable]
-h, --help Print help

5.2 示例

1. 在 MX_MAIN 分支下，使用 rua perfan 命令解析 profiling 文本中属于 d-plane 的地址：

```
> cat A3600.profile.txt
Samples | Source code & Disassembly of d-plane for cycles (7209 samples, percent: local period)
-----|-----
1 : 609ba0:      push    %rbp
2 : 609ba1:      mov     %rsp,%rbp
4 : 609ba6:      push    %r14
1 : 609ba8:      push    %r13
1 : 609baa:      push    %r12
2 : 609bac:      push    %rbx
2 : 609bad:      sub     $0xa8,%rsp
121 : 609bb4:      rdtsc
1 : 609bb9:      mov     %eax,%eax
4 : 609bbb:      lea     0x4a055be(%rip),%rdx      # 500f180 <dp_glb_vars>
1 : 609bc6:      lea     0x4a05553(%rip),%rbx      # 500f120 <flow_q_head>
4 : 609bcd:      mov     %rsi,-0xb8(%rbp)
19 : 609bd4:      or      %rax,-0xb8(%rbp)
1 : 609be2:      mov     (%rax),%rax
1 : 609be5:      mov     %rax,-0xc0(%rbp)
4 : 609bec:      lea     0x55e3add(%rip),%rax      # 5bed6d0 <dp_current_core_id>
4 : 609bf3:      mov     (%rax),%eax
4 : 609bf7:      imul    $0x61c0,%rcx,%rcx
1 : 609bfe:      lea     0x6d0(%rcx,%rdx,1),%rsi
3 : 609c06:      mov     (%rbx),%rdx
2 : 609c09:      mov     %rsi,-0xd0(%rbp)
4 : 609c1d:      prefetcht0(%rdx)
5 : 609c26:      lea     0x55e3b13(%rip),%rax      # 5bed740 <dp_glb_vars_local>
3 : 609c2d:      mov     $0x200,%esi
1 : 609c35:      mov     0x460(%rax),%r14d
2 : 609c3c:      mov     $0x20,%eax
1 : 609c44:      cmovle  %esi,%eax
2 : 609c57:      test    %r13d,%r13d
1 : 609c5a:      jg       60a3b5 <hs_flow_process_packets+0x815>
4 : 609c60:      lea     0x55e3a71(%rip),%rax      # 5bed6d8 <dp_debug_buf_data_len>
2 : 609c67:      mov     (%rax),%r12d
2 : 609c6a:      test    %r12d,%r12d
4 : 609c6d:      jne     60a3cd <hs_flow_process_packets+0x82d>
2 : 609c73:      lea     0x55e3a4a(%rip),%rax      # 5bed6c4 <debug_ctrl_flag>
```

原始 A3600 profiling 文本

```
> rua perfan -e ./bin/obj-emulator-a-dnv-ipv6-2.0/d-plane ./A3600.profile.txt
|-----| #samples:62246 | #daemons:28 | #funcs:386 | #lines:4604 | -----|

***** d-plane |percentage:94.3563%| #samples:58733/62246 | #funcs:139/386 | #lines:3965/4604 *****

Percentage      Count      Address      Instruction      Location
-----
11.5815%      7209/62246      [d-plane]
0.0016%      1/62246      609ba0      push    %rbp      hs_flow_process_packets@flow_main.c:1115
0.0032%      2/62246      609ba1      mov     %rsp,%rbp      hs_flow_process_packets@flow_main.c:1115
0.0064%      4/62246      609ba6      push    %r14      hs_flow_process_packets@flow_main.c:1115
0.0016%      1/62246      609ba8      push    %r13      hs_flow_process_packets@flow_main.c:1115
0.0016%      1/62246      609baa      push    %r12      hs_flow_process_packets@flow_main.c:1115
0.0032%      2/62246      609bac      push    %rbx      hs_flow_process_packets@flow_main.c:1115
0.0032%      2/62246      609bad      sub     $0xa8,%rsp      hs_flow_process_packets@flow_main.c:1115
0.1944%      121/62246      609bb4      rdtsc      hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:169
0.0016%      1/62246      609bb9      mov     %eax,%eax      hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170
0.0064%      4/62246      609bbb      lea     0x4a055be(%rip),%rdx      hs_flow_process_packets@flow_main.c:1122
0.0016%      1/62246      609bc6      lea     0x4a05553(%rip),%rbx      hs_flow_process_packets@flow_main.c:1124
0.0064%      4/62246      609bcd      mov     %rsi,-0xb8(%rbp)      hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170
0.0305%      19/62246      609bd4      or      %rax,-0xb8(%rbp)      hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170
0.0016%      1/62246      609be2      mov     (%rax),%rax      hs_flow_process_packets@flow_main.c:1120
0.0016%      1/62246      609be5      mov     %rax,-0xc0(%rbp)      hs_flow_process_packets@flow_main.c:1120
0.0064%      4/62246      609bec      lea     0x55e3add(%rip),%rax      hs_flow_process_packets@flow_main.c:1121
0.0064%      4/62246      609bf3      mov     (%rax),%eax      hs_flow_process_packets@flow_main.c:1121
```

A3600 profiling 文本经 rua perfan 解析后

2. 传入多个 elf 参数，解析多个二进制的地址：

```
rua perfan -e ./bin/obj-emulator-a-dnv-ipv6-2.0/d-plane -e ./bin/obj-emulator-a-dnv-ipv6-2.0/netd A3600.profile.txt
```

注意：当传入非 C 语言（包括 C++）生成的二进制中，符号是混淆过的，函数名可能比较奇怪，这是正常现象。结果中提供有文件名和行号，凭此信息能够准确地定位代码行。

5.3 输出格式

46 pending changes

-----| #samples:62246 | #daemons:28 | #funcs:386 | #lines:4604 | -----|

***** d-plane |percentage:94.3563%| #samples:58733/62246 | #funcs:139/386 | #lines:3965/4604 *****

Percentage Count Address Instruction Location

11.5815% 7209/62246 [d-plane] 所属elf hs_flow_process_packets@flow_main.c:1115

0.0016% 1/62246 609ba0 push %rbp hs_flow_process_packets@flow_main.c:1115

0.0032% 2/62246 609ba1 mov %rsp,%rbp hs_flow_process_packets@flow_main.c:1115

0.0064% 4/62246 609ba6 push %r14 hs_flow_process_packets@flow_main.c:1115

0.0016% 1/62246 609ba8 push %r13 hs_flow_process_packets@flow_main.c:1115

0.0016% 1/62246 609baa push %r12 hs_flow_process_packets@flow_main.c:1115

0.0032% 2/62246 609bac push %rbx hs_flow_process_packets@flow_main.c:1115

0.0032% 2/62246 609bad sub \$0xa8,%rsp hs_flow_process_packets@flow_main.c:1115

0.1944% 121/62246 609bb4 rdtsc hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:169

0.0016% 1/62246 609bb9 mov %eax,%eax hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170

0.0064% 4/62246 609bbb lea 0x4a055be(%rip),%rdx hs_flow_process_packets@flow_main.c:1122

0.0016% 1/62246 609bc6 lea 0x4a05553(%rip),%rbx hs_flow_process_packets@flow_main.c:1124

0.0064% 4/62246 609bcd mov %rsi,-0xb8(%rbp) hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170

0.0305% 19/62246 609bd4 or %rax,-0xb8(%rbp) hs_flow_process_packets@flow_main.c:1118->time_count_reg_get@i686_core_drv.h:170

0.0016% 1/62246 609be2 mov (%rax),%rax hs_flow_process_packets@flow_main.c:1120

0.0016% 1/62246 609be5 mov %rax,-0xc0(%rbp) hs_flow_process_packets@flow_main.c:1120

0.0064% 4/62246 609bec lea 0x55e3add(%rip),%rax hs_flow_process_packets@flow_main.c:1121

0.0064% 4/62246 609bf3 mov (%rax),%eax hs_flow_process_packets@flow_main.c:1121

0.0064% 4/62246 609bf7 imul \$0x61c0,%rcx,%rcx hs_flow_process_packets@flow_main.c:1122

0.0016% 1/62246 609bfe lea 0x6d0(%rcx,%rdx,1),%rsi hs_flow_process_packets@flow_main.c:1122

0.0048% 3/62246 609c06 mov (%rbx),%rdx hs_flow_process_packets@flow_main.c:1124

0.0032% 2/62246 609c09 mov %rsi,-0xd0(%rbp) hs_flow_process_packets@flow_main.c:1122

0.0064% 4/62246 609c1d prefetcht0(%rdx) hs_flow_process_packets@flow_main.c:1122

0.0090% 5/62246 609c26 lea 0x55e3b13(%rip),%rax hs_flow_process_packets@flow_main.c:1128

0.0048% 3/62246 609c2d mov \$0x200,%esi hs_flow_process_packets@flow_main.c:1117

0.0016% 1/62246 609c35 mov 0x460(%rax),%r14d hs_flow_process_packets@flow_main.c:1117

0.0032% 2/62246 609c3c mov \$0x20,%eax hs_flow_process_packets@flow_main.c:1117

0.0016% 1/62246 609c44 cmovle %esi,%eax hs_flow_process_packets@flow_main.c:1117

0.0032% 2/62246 609c57 test %r13d,%r13d hs_flow_process_packets@flow_main.c:1139

0.0016% 1/62246 609c5a jg 60a3b5 <hs_flow_process_packets@flow_main.c:1139>

0.0064% 4/62246 609c60 lea 0x55e3a71(%rip),%rax hs_flow_process_packets@flow_main.c:1142

0.0032% 2/62246 609c67 mov (%rax),%r12d hs_flow_process_packets@flow_main.c:1142

0.0032% 2/62246 609c6a test %r12d,%r12d hs_flow_process_packets@flow_main.c:1142

0.0064% 4/62246 609c6d jne 60a3cd <hs_flow_process_packets@flow_main.c:1142>

0.0032% 2/62246 609c73 lea 0x55e3a4a(%rip),%rax hs_flow_process_packets@flow_main.c:1143

0.0016% 1/62246 609c7a xor %r14d,%r14d hs_flow_process_packets@flow_main.c:1138

0.0048% 3/62246 609c7d mov (%rbx),%r15 hs_flow_process_packets@flow_main.c:1159

函数名@文件名:代码行
-> 内联函数名@文件名:代码行

Rua perfan 输出格式解析

5.4 工具比较

代码库中 tool/perf2func 具有相同的功能，本工具的优势在于：

- 速度更快，相比于 perf2func 提速 1500 倍左右
- 输出格式更友好，对内联函数有较好的展开处理，方便用户定位代码行

6 clean

清除工程根目录下的所有编译文件。该功能类似于最新 MX_MAIN 分支上（进行了 Makefile 优化）的 stoneos-clean。此前存在一些问题，本次完善了该功能。clean 会做以下三个工作：

1. 删除 target 文件夹
2. 删除 webui 文件夹（与分支同名）
3. 删除未受 SVN 管控的文件，改功能与 make stoneos-clean 相似。

Rua clean 允许开发者自己指定所要保留的文件（通过配置文件或命令行参数）。

6.1 用法

用户可以通过 `rua init -h` 查看帮助信息：

```
> rua clean -h
Clean build files (run under project root)

Usage: rua clean [OPTIONS] [ENTRY]...

Arguments:
  [ENTRY]...  Files or dirs to be cleaned ('target' is always included even if
not specified)

Options:
  -n, --ignore <FILE>  File or directory to be ignored while cleaning. You can
add multiple ignores by specifying this option multiple times
  -h, --help            Print help

Examples:
  rua clean # Clean the entire project

Caution:
  All unversioned files will be REMOVED permanently, including files created by
YOU but not
  added to SVN. Use it carefully!
```

6.2 示例

1. 清除工程根目录下的所有编译文件：

```
rua clean .
```

注意：该命令会删除当前目录下的所有编译文件，包括 webui 文件夹和 target 文件夹

2. 清除工程根目录下的所有编译文件，但保留文件夹 .rua、.cache 和文件 .ignore 和 compile_commands.json：

```
rua clean -n .rua -n .cache -n .ignore -n compile_commands.json .
```

```
bzhao in build18-bj in ~/repos/MX_MAIN
> touch aaa
bzhao in build18-bj in ~/repos/MX_MAIN
> svn status .
?      .cache
?      .ignore
?      .rua
?      aaa
?      compile_commands.json
M      sdk/phytium-kylin-sdk/linux/kernel/linux
M      src/d-plane/flow/flow_first.c
M      src/d-plane/policy/dp_policy.c
bzhao in build18-bj in ~/repos/MX_MAIN
> rua clean -n .cache -n .rua -n .ignore -n compile_commands.json
[1/3] Removing target objs...ok
[2/3] Removing WebUI objs...ok
[3/3] Removing unversioneds...ok
bzhao in build18-bj in ~/repos/MX_MAIN
> svn status .
?      .cache
?      .ignore
?      .rua
?      compile_commands.json
M      sdk/phytium-kylin-sdk/linux/kernel/linux
M      src/d-plane/flow/flow_first.c
M      src/d-plane/policy/dp_policy.c
```

aaa 被清除

3. 在 ~/.rua/config.toml 或 \$workspace/.rua/config.toml 中添加下面的配置内容，rua clean 在执行时会自动忽略这些文件：

```
[clean]
ignores = ["compile_commands.json", ".cache", ".rua", ".ignore"]
```

```
bzhao in build18-bj in ~/repos/MX
> svn status .
?      .cache
?      .ignore
?      .rua
?      aaa
?      compile_commands.json
?      src/mgmt/webserver/nginx/Makefile
?      target
bzhao in build18-bj in ~/repos/MX
> rua clean .
[1/3] Removing target objs...ok
[2/3] Removing WebUI objs...ok
[3/3] Removing unversioneds...ok
bzhao in build18-bj in ~/repos/MX
> svn status .
?      .cache
?      .ignore
?      .rua
?      compile_commands.json
bzhao in build18-bj in ~/repos/MX
```

7 init

rua 可以在 bash/zsh/fish 等 shell 中自动补全。

7.1 用法

用户可以通过 `rua init -h` 查看帮助信息：

```
> rua init -h
Generate completion for the given shell

Usage: rua init <SHELL>

Arguments:
  <SHELL>  Shell type [possible values: bash, elvish, fish, powershell, zsh]

Options:
  -h, --help  Print help

Note:
  eval "$(rua init bash)" # Append this line to ~/.bashrc
  eval "$(rua init zsh)"  # Append this line to ~/.zshrc
```

7.2 示例

1. 在 bash 中自动补全：

```
eval "$(rua init bash)"
```

2. 在 zsh 中自动补全：

```
eval "$(rua init zsh)"
```

3. 在 fish 中自动补全：

```
eval (rua init fish)
```

4. 在 powershell 中自动补全：

```
rua init powershell | Out-String | Invoke-Expression
```

5. 在 elvish 中自动补全：

```
eval (rua init elvish)
```